

2026-09-30

Native GeoArrow Computation Engine in R

Josiah Parry 

This template was last revised on 2026-03-24.

Executive Summary

This proposal seeks to ameliorate three challenges facing the R-spatial ecosystem.

First, spatial analysis in R “is slow” and can be dramatically improved. The de facto package for vector-based spatial analysis in R is the `{sf}` package. Due to its ergonomic API and its integration with the tidyverse, it is the common choice for spatial data analysis. However, `{sf}` can struggle to scale effectively and has caused developers to turn to other tools and other languages.

Secondly, the past few years have seen the rapid adoption of Apache Arrow—a standard for in-memory representation of arrays and tables. The spatial ecosystem, in particular, has been quick to adopt GeoArrow, an extension to the Arrow standard that defines how spatial data can be represented with Apache Arrow.

The standardization on (Geo)Arrow enables the development of packages that can work on in-memory data without incurring any additional overhead costs of deserializing and serializing data into intermediate objects.

Lastly, concurrent to the development of GeoArrow has been the development of GeoParquet. At present, GeoParquet support in R has focused on consuming the format. To my knowledge, there is no R package that provides tailored support for writing GeoParquet as well as reading from remote storage.

I seek \$5,000 USD to fund the completion of the development of a new Rust-based, GeoArrow native, and highly performant spatial computation engine for R called `{geoarrows}`. The R package will provide:

- GeoArrow-native and parallelized computation engine
- Persistent spatial RTree and KDTree spatial indexes for efficient spatial querying and joining
- Drop in compatibility with `{sf}`
- Enhanced GeoParquet support such as spatial sorting, bounding boxes on row-chunks for predicate pushdown, and persisting coordinate reference systems.

Signatories

Project team

Josiah Parry is a Sr. Product Engineer in Spatial Statistics at Esri.

Contributors

Consulted

The Problem

The (de)serialization problem

While `{sf}` is a foundational package for the R ecosystem, its performance has failed to scale effectively as the size of data that people commonly work with has grown. This is a common experience. Users [report slow distance calculations on modest data](#), [write their own batching workarounds to finish jobs at all](#), and [performance](#) remains an open topic in `{sf}`'s own issue tracker.

The biggest challenge with `{sf}` is primarily the fact that geometries are stored in R as a list. Each operation that `{sf}` performs has to be first converted to a GEOS or S2 geometry. The S2 or GEOS libraries perform operations on their native geometries and then `{sf}` captures their results and again converts them back to their R representation. This limitation has led GDAL to create a GeoArrow native “column-oriented” reader which uses Apache Arrow natively to improve performance. This process of converting to and from is the expensive (de)serialization that the GeoArrow standard exists to partially help address.

GeoArrow as a standard exists to address exactly this problem. It has been a focus of [Spatial Data Science Across Languages](#) since its inception in 2024 and [again in 2025](#), and as a result we are seeing coordinated adoption across all languages. As an indicator of its importance, GDAL added a [column-oriented Arrow API](#). GeoPandas, too, added [native GeoArrow support](#).

At present, R support is limited. The R package `{geoarrow}` provides a `{nanoarrow}` and `{arrow}` compatible vector representation. The ISC-funded `{geoarrowWidget}` package assists in transporting `{geoarrow}` vectors to the browser. However, the R community lacks a GeoArrow-native compute engine that can scale effectively.

Alternative Computation Engines

`{geos}` has historically been the fastest R package in the ecosystem but is very low level and thus not very widely used.

Currently, there are two primary options for scalable geospatial computation engines in R: 1) `{duckspatial}` and 2) `{sedonadb}`. Both of these face similar issues: neither work with native `data.frames` and both require external downloads. `sedonadb` is especially challenged due to not being on CRAN.

The `duckspatial` R package uses the [spatial extension](#) to DuckDB. DuckDB relies on downloading and installing an extension. The size alone is a reason for concern. Secondly, it requires [static linking](#). `{sedonadb}` provides bindings to [Apache Sedona](#). The most fit for large scale analysis but is not available on CRAN and lacks sufficient documentation as well as has a bespoke API. The latter, while arguably the most scalable package, is not available on CRAN and lacks sufficient documentation leading to limited adoption.

GeoParquet

- GeoParquet is becoming the standard for storing and sharing spatial data, particularly on cloud object storage
- The {geoarrow} R package provides GeoParquet metadata to the {arrow} R package so that {arrow} can read GeoParquet natively.
- Writing GeoParquet is can be done via `arrow::write_parquet()` (note that sfarrow delegates to arrow directly)
- This is minimally viable GeoParquet.
- Parquet's benefit is that row-group metadata can be used to efficiently filter data.
- At present there is no way easy way to record spatial metadata in a GeoParquet file
- Geometries are written in the order that they are in the data.frames
- Spatial proximity is entirely ignored and thus row-groups are not are not spatially organized meaning libraries cannot spatially filter data without ingesting it.
- lastly, there is no projjson support in any existing library which means that the CRS of our data can be dropped and downstream tools will not know its CRS
- The result is a file that other tools can open but that has lost its CRS, and that forces a full scan to answer a spatial query — which on remote storage means transferring the entire file

we need an R package with a modest footprint, no system dependencies, no external downloads, that works on GeoArrow natively, provides fast, scalable, and efficient spatial operations, as well as true GeoParquet support.

The proposal

Overview

The {geoarrowr} package is a GeoArrow-native spatial computation engine designed for ergonomic use within the R ecosystem. The package is in the middle of its development life cycle and this grant would fund its completion.

Today, the package provides:

- performant readers for shapfile, GeoJson, and FlatGeobuf
- R-tree and KD-tree spatial indexes
- Spatial, nearest neighbor, and distance band joins
- User defined functions registered with {arrow}

This package aims to be faster than geos, competitive with duckspatial and sedonadb, and as ergonomic as {sf} (with a drop in replacement via `mask_sf()`) for **in memory** computation.

Completion of the package is split into four milestones:

1. Complete core geometry engine so that every algorithm provided by the Rust crate `geo` as of this writing is available from R. This would achieve near parity with {sf} on most operations.
2. Create spatial indexes that can be written to disk which can be used to spatially sort geometries for more efficient GeoParquet storage.
3. Provide drop in {sf} compatibility via `mask_sf()` so that {geoarrowr} can be used as a backend for {sf} and release the package on CRAN.

4. Persist coordinate reference systems, so that GeoArrow transfer between R, Python, DuckDB, and GDAL can be lossless. This milestone also targets GeoParquet column meta persistence enabled by milestone 2.

Prior art

The `{geoarrowr}` package is the culmination of years of exploring the intersection of Rust, geospatial, and GeoArrow. Initially, the `{rsgeo}` package was created to be a zero-dependency and high-performance spatial computation engine with [Rust based conversion](#) to and from `{sf}`.

During this time GeoArrow began to pick up. With the `{anime}` line matching algorithm being a proof of concept for use of GeoArrow in R built on my [arrow-extendr](#) Rust library which provides FFI between R and Rust for `{arrow}`, `{nanoarrow}`, `{polars}`, and `{geoarrow}` packages.

In 2024, the [initial version](#) of this package was started. But development was paused due to an unstable `geoarrow-rust` API, and challenges with Arrow FFI that are now resolved.

Detail

Minimum Viable Product

A minimal viable product provides the core geometry implementations on `geoarrow` arrays in R. The state of the R package today is nearly an MVP. With this funding it can be improved to be a high quality spatial package for the R ecosystem.

Architecture

The `{geoarrowr}` package is a Rust-powered R package built on [extendr](#). [Arrow](#) to Rust FFI is handled via the [arrow-extendr](#) crate that I built and maintain. The R representation of `GeoArrow` arrays leans on the implementation of the `{geoarrow}` R package. Native algorithms use the `geo` Rust crate via the constant-time access via the `geo-traits` crate. Parallelization of geometric operations is handled via the `rayon` crate. In GeoArrow, coordinate reference systems (CRS) must be represented as [projjson](#). For this, a new Rust crate, [projjson](#), was prototyped and published. Milestone 4 integrates it into `{geoarrowr}`. Spatial indexes are created and persisted using the [geo-index](#) Rust crate.

Assumptions

There are no outstanding assumptions that can invalidate the concept proposed here. It has been validated over three years of experimentation and development in the geospatial and Rust environments.

External dependencies

`{geoarrowr}` has no external system dependencies. It does not link GDAL, GEOS, PROJ, DuckDB, nor are there any runtime downloads. Dependencies are pure-Rust that are only required at build time. CRAN will build and provide package binaries.

- Imports: `cli`, `rlang`
- Suggests: `arrow`, `dplyr`, `geoarrow`, `nanoarrow`, `sf`, `wk`, `testthat`, and `withr`
- Rust and Arrow FFI: `extendr-api`, `arrow-extendr`, and `arrow`
- Geometry operations: `geo`, `geo-traits`, and `geo-index`
- GeoArrow array representation: `geoarrow`, `geoarrow-array`, `geoarrow-cast`, `geoarrow-flatgeobuf`, and `geoarrow-geojson`
- File format readers: `flatgeobuf`, `geojson`, and `shapefile`

- Parallelism and error handling: [rayon](#), [serde_json](#), and [anyhow](#)

Limitations

There are 3 primary limitations.

1. Rust powered R packages published to CRAN must vendor their dependencies' source code. As a result, the R package tar ball may exceed CRAN's limits. If this occurs, the R package will need to be partitioned into multiple smaller packages each individually published and `geoarrowrs` could become a metapackage to load and reexport all of these.
2. The core library for geometric operations works only on 2D geometries meaning any operations on geometries with Z or M values is silently lost.
3. There is limited support for `projjson` in the R ecosystem. Converting to and from `projjson` will be addressed as part of milestone 4. Handling of *existing* `projjson` will be straight forward. Conversion from `{sf}` `crs` objects may prove to be somewhat challenging.

Project plan

Start-up phase

Technical delivery

- Core Algorithm Implementation
- Spatial Joins
- Parallelization and optimization
- `{arrow}` registered
- pkgdown site
- `spatialbench` benchmark
- `duckspatial` benchmark
- CRAN release

Other aspects

- i'd like to submit the package to rOpenSci or R-Spatial if they'd take it
- Social media posts
- Joint Blog post with R Consortium

Budget & funding plan

I request \$5,000 USD to fund the development of `{geoarrowrs}`.

Milestone 1 - Complete Core Geometry Engine Implementation (\$1,500)

This milestone funds the development effort to complete the core computation engine in `{geoarrowrs}` such that there is near-complete parity with `{sf}`. To accomplish the milestone, the following will be completed:

- Geometry validation: `st_is_valid()`
- Geometry reparation: `st_make_valid()`
- Geometry type checking: `st_geometry_type()` and `st_is()`
- Collection extraction: `st_collection_extract()`
- Perimeter calculation: `st_perimeter()`
- Coordinate comparison: `st_equals_exact()`
- Jittering: `st_jitter()`
- Node: `st_reverse()`, `st_node()`
- Areal interpolation: `st_interpolate_area()`
- Bounding box: `st_bbox()`
- Polygon constructors:

The following features are out of scope of this proposal as there are no pure rust alternatives to rely on at present.

- 3D geometries
- Snapping
- Polygonization
- Geometry Projection

Milestone 2 - Persistent Spatial Indexes and Spatial Sorting (\$1,000)

This milestone funds the development of creating and persisting spatial RTree and KDTree spatial indexes. These spatial indexes are created on the fly for each operation performed by `{sf}` as well as other libraries. For operations that require repeated look ups, persisting the index can be highly efficient.

Further, these indexes can be used to perform sorting of geometries to ensure that nearby geometries are ordered together. Spatial sorting can greatly improve the effectiveness of row group statistics in GeoParquet files. Partial reads of a GeoParquet file can leverage the sorting more effectively.

For spatial sorting, Hilbert and sort-tile-recursive (STR) will both be supported. `{geos}` only provides STR sorting and spatial RTrees, to my knowledge, do not have support in the R ecosystem.

Milestone 3 - `{sf}` Compatibility and CRAN release (\$1,500)

In order for the package to be useful to the broadest community, this milestone will create drop in support for `{sf}` via S3 namespace masking. Users can call `mask_sf()` to overload the `{sf}` namespace with generics that enable `{sf}` to work with `{geoarrowr}` geometries.

Following, documentation, examples, and benchmarks will be tidied up, and rewritten for ease of adoption.

One **risk** is that the source tarball is 14mb which is 9mb larger than the requested CRAN limit. However, exceptions can be made. For example, the prior art `{rsgo}` was a [9mb tarball](#). And other Rust based R packages such as `{fcl}`, are [15mb](#) and supported on CRAN.

One **limitation** is that the `geo` crate that we rely on geometry operations does not have full spherical support whereas `{sf}` does via the `{s2}` package. In the event that a spherical CRS is used

Milestone 4 - GeoArrow and GeoParquet metadata

At present, `geoarrowr` cannot propagate a coordinate reference system (CRS) across the R to Rust FFI boundary. Milestone 3 is possible by leaning on `{sf}`'s CRS object and adding it as an attribute on the `geoarrow` vectors which requires PROJ to be installed. This milestone would ensure that no external system dependencies would be needed.

Further, the GeoArrow specification records CRS as column level metadata in the projjson format. To support this, the `projjson-rs` crate was created which provides strongly typed reading and writing of `projjson` with no external system dependencies.

As a result, GeoArrow transferred from DuckDB, R, Python, and GDAL would propagate the CRS correctly.

The column level metadata is also utilized by GeoParquet. As such, this milestone completely implements the GeoParquet column metadata including:

- encoding
- geometry_types
- crs
- bbox
- orientation
- edges
- epoch

Notably, the `edges` indicates wh

Success

Definition of done

Measuring success

Correctness tests with sf

Future work